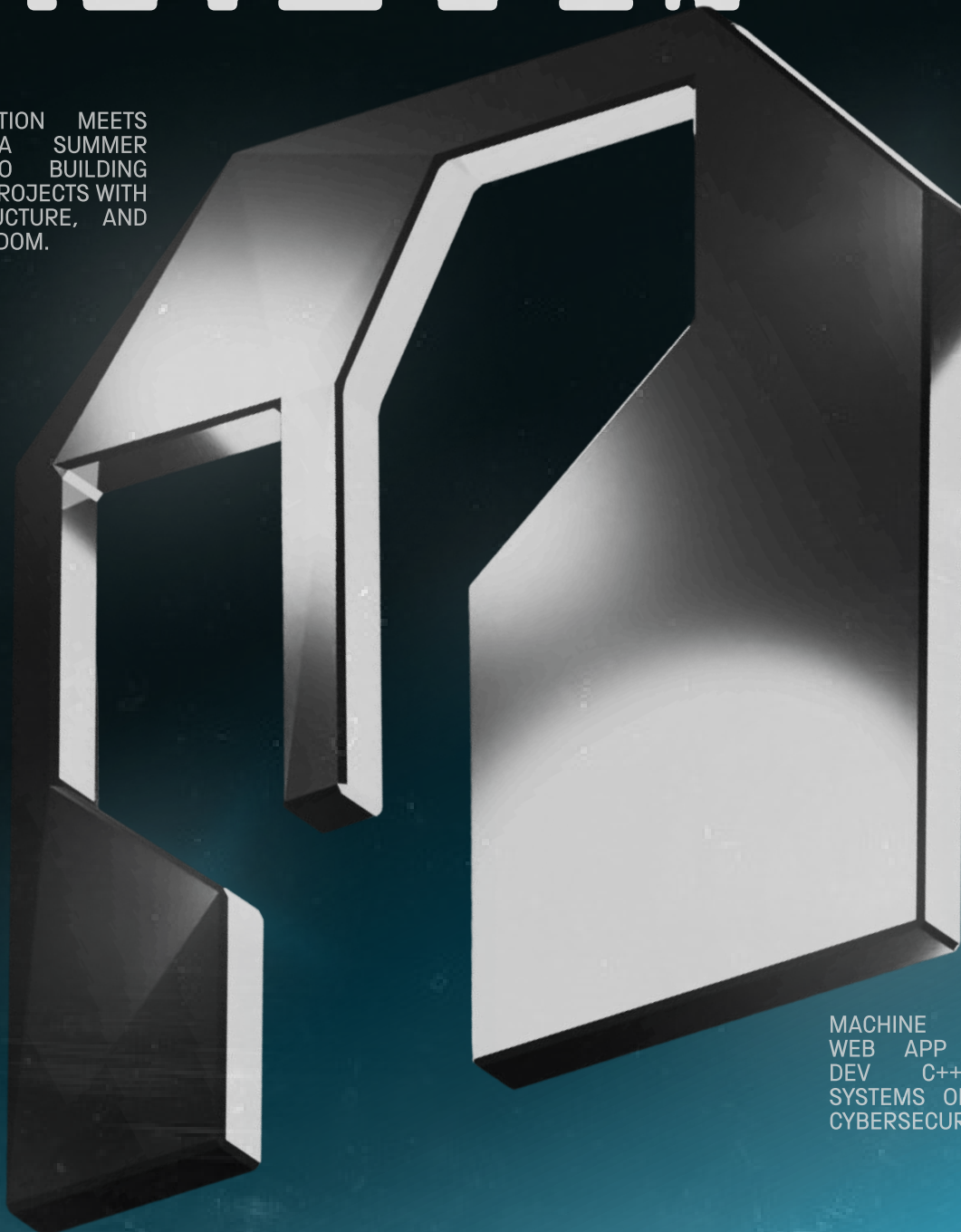


EVEN SEMESTER PROJECTS



WHERE AMBITION MEETS
EXECUTION. A SUMMER
DEDICATED TO BUILDING
MEANINGFUL PROJECTS WITH
CLARITY, STRUCTURE, AND
CREATIVE FREEDOM.



MACHINE LEARNING
WEB APP SOFTWARE
DEV C++/OPERATING
SYSTEMS OPTIMISATION
CYBERSECURITY & MORE

CODING
CLUB

14TH
APRIL '26

IIT
GUWAHATI

TABLE OF CONTENTS

INTRODUCTION	3
GRADEOPS	4
VIRTUAL-LAB:	5
AUTOLYRICS	6
CINEIQ	8
FAILSAFE	9
CVM++	10
PHISHGUARD	12
GLASSBOARD	14
HACKATHON SQUAD	16

INTRODUCTION



Welcome to a summer dedicated to building meaningful projects with clarity, structure, and creative freedom. These projects are specifically designed for students with **beginner-level skills** in their respective domains who want to gain hands-on experience by solving real-world problems. Whether you are interested in **Machine Learning, Web and App Development, Software Systems, Cybersecurity, Data Science, or Algorithms**, there is a project here to help you bridge the gap between theory and practice.

Completing these projects will provide a significant boost to your **resume**, showcasing your ability to deliver functional solutions such as automated grading pipelines, symbolic math libraries, or phishing detection systems. To assist you along the way, we encourage you to join our **WhatsApp community** for doubt clearing, clarifications, and direct interaction with your project mentors.

KEY DATES

Release Date: 14th April 2026.

Submission Deadline: 14th May 2026.

JOIN THE COMMUNITY

[Link to Community.](#)

GRADEOPS



Grading handwritten exams is time-consuming, inconsistent, and prone to fatigue-induced bias. To solve this, develop a Human-in-the-Loop (HITL) grading pipeline that uses Vision-Language Models (VLMs) and Agentic LLMs to evaluate scanned exams against strict rubrics. The system will allow professors to upload bulk exam scans and JSON rubrics. It will automatically extract, transcribe, and grade complex handwritten answers, awarding partial credit with justifications. These AI-proposed grades are then pushed to a high-speed dashboard, allowing Teaching Assistants (TAs) to rapidly review, approve, or override the decisions, ensuring fairness and massive time savings.

GOALS:

- A web portal to upload bulk exam scans (PDFs) and define granular grading rubrics.
- Role-Based Access Control (Instructors vs. TAs).
- Specialized OCR/Vision models to extract messy handwritten answers and store them in cloud storage.
- An Agentic LLM pipeline to:
 - Award partial credit based on strict rubric conditions.
 - Generate structured textual justifications for the score.
 - Flag highly similar logic structures across papers for plagiarism.
- A high-throughput review dashboard showing the cropped student answer side-by-side with the AI grade, featuring keyboard shortcuts for rapid TA approvals or overrides.

TECHSTACK/FRAMEWORKS :

- Machine Learning : Python, PyTorch, Hugging Face (Nougat/Qwen-VL for OCR), Langchain and Langgraph.
- Web : HTML/CSS + React.js, FastAPI or Node.js, PostgreSQL or MongoDB.

MENTORS:

ABHINAV RAI 7004772095

Teaching complex physics and engineering concepts online is often limited to static videos and non-interactive text, failing to build intuition for dynamic physical systems. VIRTUAL-LAB addresses this by providing a "Digital Twin" environment—a collaborative 2D physics sandbox designed for university-level learning. The platform allows multiple users to build machines, test structural integrity, and observe real-time forces in a shared, high-fidelity workspace, effectively bridging the gap between theoretical equations and physical reality through hands-on experimentation.

DELIVERABLES:

- Interactive Physics Canvas: A web-based workspace allowing users to drag, drop, and configure physical bodies, shapes, and materials.
- Multi-User Room Engine: A backend system managing synchronized states across users to ensure a seamless, shared physical experience in real-time.
- Physics Accuracy & Constraint System: Integration of a 2D physics engine (Matter.js) with a functional UI toolset for creating mechanical connections like ropes, springs, pivots, and motorized components.
- Real-Time Analytics Dashboard: An integrated panel that generates live line charts and vector arrows to visualize velocity, kinetic energy, and forces acting on specific components.
- Experiment Library: A gallery view where users can browse, save, share, and load pre-configured physics scenarios or "lab templates" for classroom assignments.
- Agent Middleware: A high-frequency synchronization layer that broadcasts physics engine deltas to minimize network lag and resolve state conflicts between collaborators.

TECHSTACK/Frameworks :

- Frontend: React.js, Matter.js (Physics Engine), Tailwind CSS
- Backend: Node.js, Express.js
- Database: MongoDB
- Real-time: WebSockets (Socket.io)
- Data Visualization: Recharts or Chart.js

AUTOLYRICS



Standard Automatic Speech Recognition (ASR) systems are primarily designed for spoken language and often perform poorly on singing due to pitch variations, prolonged phonemes, and background instrumentation. This leads to inaccurate or incomplete lyric transcriptions when applied to music. Transform an open-source ASR Transformer model into a system capable of better transcribing singing audio. Improve its performance through lightweight fine-tuning techniques and evaluate how well it adapts to the challenges of music-based inputs compared to its original pre-trained version.

GOALS

- Ingest a short music clip and generate a transcription that:
 - Captures sung lyrics more accurately than the base model without relying on external source-separation.
 - Handles variations in pitch and rhythm present in singing.
 - Produces readable and structured text output.
- Compare model performance across different approaches on a highly curated, constrained subset of singing-voice data:
 - Zero-shot inference (pre-trained baseline).
 - Parameter-Efficient Fine-Tuning (LoRA applied to the decoder).
 - (Optional) LoRA applied to both the encoder and decoder.
- Performance Report (PDF) including:
 - Dataset description, curation methodology, and train/val/test splits.
 - Comparative results using metrics like Word Error Rate (WER) and Character Error Rate (CER). (Target: >15% relative WER reduction vs. baseline).
 - Inference latency and resource usage (VRAM).
 - Summary of improvements and trade-offs between approaches.

TECHSTACK/FRAMEWORKS :

- Machine Learning : Python, PyTorch, Torchaudio
- Transformers & Fine-Tuning : HuggingFace Transformers, HuggingFace Datasets, PEFT, BitsAndBytes
- Evaluation & Deployment : Jiwer (WER/CER metrics), Gradio (interactive demo)

MENTORS:

MANTHAN 9512529995

NILAY 9340251202

DATASET SOURCES

1. DALI (Dataset of Aligned Languages and Melodies for Singing): It contains over 5,000 polyphonic songs with time-aligned lyrics.
2. NUS Sung and Spoken Lyrics Corpus (NUS-48E): A much smaller, highly controlled dataset containing 48 English songs. It features audio of subjects both speaking and singing the same lyrics.
3. Jamendo Lyrics Dataset: Contains polyphonic music with lyric annotations, specifically built for audio-to-lyrics alignment and transcription.
4. Hugging Face Hub: Search the Hugging Face datasets library for tags like singing, acapella, or lyrics.

PROBLEM STATEMENT

Content discovery on modern streaming platforms is opaque, biased toward promoted titles, and traps users in recommendation loops. There is a need for an open, explainable movie recommendation engine that combines multiple ML strategies to deliver personalized, interpretable suggestions that evolve with user taste over time.

DELIVERABLES

- Hybrid Recommendation Engine: Combines collaborative filtering, content-based filtering (TF-IDF + cosine similarity), and SVD-based matrix factorization via a weighted ensemble
- Sentiment-Aware Re-Ranker: Uses VADER/DistilBERT on user reviews to re-rank recommendations based on real audience reception signals
- User Taste Dashboard: Streamlit interface visualizing genre radar charts, decade preferences, and director/actor affinities from rating history
- Explainability Layer: Every recommendation surfaces a human-readable reason using LIME or rule-based templates

DATASETS:

- MovieLens 25M — grouplens.org/datasets/movielens/25m
- TMDB Metadata (Kaggle) — cast, genres, keywords for 45K movies
- IMDB 50K Reviews (Kaggle) — for sentiment model training

TECH STACK:

- ML: Python, scikit-learn, Surprise (SVD), Pandas, NumPy
- NLP: VADER / HuggingFace DistilBERT
- Serving: FastAPI (/recommend and /similar endpoints)
- Dashboard: Streamlit + Plotly
- Tracking: MLflow for experiment logging

FAILSAFE



In educational institutions, student failure often goes undetected until end-of-semester results, leaving no room for meaningful intervention. Faculty lack a proactive, data-driven tool to identify at-risk students early and understand the root causes behind their struggles. To address this, develop a web-based system where faculty upload student data and a trained ML model predicts failure risk — with Explainable AI making every prediction transparent — and auto-generates personalised intervention plans to help faculty act before it's too late.

GOALS

- Predict at-risk students early using attendance, assignments, and behavioural data — not just final grades.
- Use SHAP to explain why each student is flagged, making predictions trustworthy and actionable for non-technical faculty.
- Auto-generate personalised interventions per student such as extra classes, counselling referrals, or study plan adjustments.
- Provide a dashboard for faculty and HODs to track risk trends, apply interventions, and monitor improvement over the semester.

DATASETS:

- Student Performance Data Set by UCI - available on Kaggle

TECH STACK:

- Machine Learning: Python, XGBoost, scikit-learn, SHAP, Pandas, Matplotlib, Seaborn
- Web: HTML/CSS + React.js, FastAPI, PostgreSQL, JWT Authentication

MENTORS:

AVNI JHALANI 6378972643

Stack-Based Virtual Machine & Custom Compiler

Problem Statement: Most developers use high-level languages like Python, JavaScript, or Java without deeply understanding how raw text is translated into instructions that a computer can actually execute. This project demystifies the compilers, bytecode, and runtime environments by building them from scratch. This project aims to build a lightweight, custom scripting language in C++ that compiles down to a proprietary bytecode, which is then executed by a custom-built, stack-based Virtual Machine (VM).

GOALS

- Design a simple language grammar and an Instruction Set Architecture (ISA).
- Build a Lexer to convert raw source code strings into Tokens (e.g., NUMBER, PLUS, IDENTIFIER).
- Build a Parser to arrange Tokens into an Abstract Syntax Tree (AST).
- Implement a Compiler to flatten the AST into an array of raw Bytecode instructions (Opcodes).
- Build a Virtual Machine (VM) with a stack-based execution loop to execute the Bytecode.

DELIVERABLES

- C++ program/library with:
- Lexer and Parser modules.
- Bytecode Compiler module.
- Virtual Machine execution engine.
- CLI-based REPL (Read-Eval-Print Loop) or File Runner demo:
- Input custom language script (e.g., script.cvm).
- Show the generated AST (optional debug mode).
- Show the compiled Bytecode output.
- Print the final execution result from the VM.
- Sample test scripts demonstrating language features.

TECHSTACK / FRAMEWORKS:

- Language: C++
- Build Tool: CMake (optional)
- Concepts:
 - Lexical Analysis (Tokenization)
 - Recursive Descent Parsing and Abstract Syntax Trees (AST)
 - Bytecode generation & Operation Codes (Opcodes)
 - Stack-based execution architecture
- Low-level memory management and fast dispatch loops (e.g., utilizing `std::vector<uint8_t>`)

RESOURCES / REFERENCES:

- Crafting Interpreters by Robert Nystrom (The definitive guide for this project architecture)

TUTORIALS

- "Writing a Lexer in C++"
- "Understanding Stack-Based Virtual Machines"

SCOPE (KEEP IT SIMPLE):

- Support basic data types: Integers, Booleans.
- Support simple operations: +, -, *, /, ==, <.
- Support basic variables and assignment (e.g., let x = 10).
- Support basic control flow (e.g., simple if/else or while loops).
- Provide a built-in input keyword to take inputs and a print keyword to see outputs.

An ML-Powered Phishing URL Detection System with Cyber Threat Intelligence Integration

Phishing attacks frequently bypass traditional static blacklists by utilizing newly registered domains. PhishGuard addresses this vulnerability by combining statistical Machine Learning classification with active Cyber Threat Intelligence.

Designed to mirror real-world Security Operations Center (SOC) workflows, the system operates as a real-time Chrome browser extension. It analyzes visited URLs on the fly, decoding obfuscation, querying live threat feeds (like VirusTotal and URLhaus), performing passive DNS/WHOIS forensics, analyzing typosquatting and implementing other methods to identify the malicious URL. Results are synthesized into structured threat reports accessible via an analyst dashboard.

GOALS

- Build & deploy an ML pipeline ($\geq 95\%$ accuracy on balanced dataset, sub-500ms response time for browser integration).
- Implement active threat intelligence: URL obfuscation decoding, live DNS/WHOIS lookups, and typosquatting detection.
- Deliver a functional Chrome extension for real-time protection, coupled with a React-based web dashboard for security analysts (scan history, exportable threat reports).

DELIVERABLES:

- Serialized ML Model: Trained classification model with documented evaluation metrics.
- REST API Backend: Deployed FastAPI application orchestrating URL feature extraction, model inference, and live CTI lookups.
- Analyst Dashboard: Web UI displaying scan history, risk distribution visuals, and downloadable structured IoC reports.
- Project Report: a brief report stating your architecture and the methods implemented.

DATASET RESOURCES:

- PhishTank: Verified, daily-updated phishing URLs
- URLhaus (API): Live malicious URL database
- Tranco Top 1M: Research-grade popular legitimate domains
- ISCX URL 2016: Kaggle benchmark dataset

TECH STACK:

- Machine Learning: Python, scikit-learn, XGBoost, Pandas
- Backend API: Python, FastAPI
- Threat Intelligence: VirusTotal API, URLhaus API
- Forensics / Core: python-whois, dnspython, urllib
- Frontend UI: React, Tailwind CSS, Recharts
- Client Extension: JavaScript, HTML/CSS

Large organizations struggle with cross-department visibility, but the critical bottleneck is cascading delays during handoffs. When Module A delays Module B, which delays Module C, traditional systems obscure the root cause, leading to finger-pointing. GlassBoard solves this by acting as a transparent dependency tracker. It enforces a verifiable "handshake" protocol for inter-team handoffs—complete with timestamps and proof. By mapping these transitions, leadership can instantly pinpoint exactly where a delay originated, while utilizing tiered privacy to keep sensitive data secure.

GOALS

- Internal Task Checklists – Provide a lightweight task management tool within each module. This allows team members to break down their deliverables into actionable steps, ensuring all requirements are met and checked off before initiating the final cross-module handshake.
- Module Progress Tracking – Allow individual departments/modules to update, visualize, and broadcast their current progress to the rest of the organization.
- Digital Handshake System – A strict handover mechanism between modules (e.g., Module A delivers to Module B). It mandates an exact timestamp, visual proof of delivery (like a photo or document), and requires an explicit "Accept" or "Reject" response from the receiving module.
- Tiered Privacy & Management View – Implement role-based access control (RBAC). Module members only see what is relevant to them, while organization heads have unrestricted, bird's-eye visibility into all modules.
- Inter-Module File Sharing & Version Control: A dedicated, collaborative workspace for files shared between dependent modules. Whenever a file (like a spec sheet or blueprint) is modified during or after a handoff, the system maintains a complete, viewable audit trail. Users can see the exact edit history, including who made the change, what was altered, and the timestamp, ensuring absolute transparency over shared assets.
- Optional Features – Push notifications for pending handshake requests and an audit log for all cross-module transactions.

GLASSBOARD



DELIVERABLES:

- Fully functional Android or iOS Application.
- Final Project Report detailing system architecture, database schema, and security protocols.

TECHSTACK/FRAMEWORKS:

- Frontend: React Native or Flutter (Android or iOS support)
- Backend: Firebase (Authentication, Firestore, Storage)
- Realtime Updates: Firebase Realtime Database
- Notifications: Firebase Cloud Messaging (FCM) for push notifications.

MENTORS:

NILOTPAL 6307064720

HACKATHON SQUAD



Your university is hosting its flagship annual hackathon, and you want to assemble the ultimate dream team from a massive pool of freshman coders. Each coder has a specific "Skill Rating" based on their coding speed and problem-solving abilities, which contributes to the team's overall strength.

However, due to past rivalries and department drama, certain pairs of coders refuse to work together. As the team captain, your task is to design an algorithm to select a subset of coders that maximizes the total skill rating of the squad without including any conflicting pairs.

Due to the massive scale of the student body, finding the absolute perfect mathematical team might be computationally impossible. Your goal is to apply optimization techniques to find a valid team with the **highest possible** total skill rating within the time limit.

GOALS

- Total Score: Sum of the skill ratings of all selected coders.
- Conflict-Free Constraint: The selected team must be strictly conflict-free; no two coders in the selected subset can have a predefined rivalry.
- Team Size: A team can consist of any number of coders, from just one individual to the entire available pool.
- Objective: Design and implement a heuristic or approximation algorithm that rapidly evaluates valid combinations and outputs the highest-scoring subset it can find within the 5-minute execution window.

INPUT FORMAT

- The first line contains two integers N and M — the total number of available coders and the number of conflict pairs.
- The second line contains N space-separated integers: $S_1, S_2, \dots, S_n$, where S_i represents the skill rating of the i -th coder.
- The next M lines each contain two integers: u and v , indicating a mutual conflict between coder u and coder v .

HACKATHON SQUAD



OUTPUT FORMAT

- The first line must contain a single integer — the total sum of skill ratings for your selected, conflict-free team.
- The second line should list the space-separated indices of the selected coders in ascending order to prove the team composition.
- (Note: Your solution will be judged based on the validity of your team and how high your score is relative to the best known solutions.)

CONSTRAINTS

- $1 \leq N \leq 200,000$
- $0 \leq M \leq (N * (N - 1)) / 2$
- $1 \leq S_i \leq 1,000,000,000$
- $1 \leq u, v \leq N$
- $u \neq v$
- All conflict pairs (u, v) are distinct.